

# CAPSTONE PROJECT

## Complete Requirement

Semester 5

---

### 60-DAY FULL STACK DEVELOPMENT CAPSTONE PROJECT

Track: Java (Spring Boot) OR Python (Django / FastAPI)

# TABLE OF CONTENTS

|                                                                        |    |
|------------------------------------------------------------------------|----|
| TABLE OF CONTENTS .....                                                | 2  |
| 1. About This Capstone Project.....                                    | 4  |
| 1.1 Purpose.....                                                       | 4  |
| 1.2 The Four Non-Negotiables .....                                     | 4  |
| 1.4 Roles .....                                                        | 4  |
| 2. Master Timeline — Day 1 to Day 60 .....                             | 5  |
| 2.1 Phase Map .....                                                    | 5  |
| 2.2 Weekly Sprint Breakdown (9 Sprints) .....                          | 5  |
| 3. Day 1 — Problem Statement Finalization .....                        | 6  |
| 3.1 Problem Statement Selection .....                                  | 6  |
| 3.2 Problem Statement Template (copy into Problem_Statement.md) .....  | 6  |
| 3.3 Acceptance Criteria .....                                          | 6  |
| 3.4 Straight Rejection List .....                                      | 7  |
| 4. Tech Stack.....                                                     | 7  |
| 4.1 Non-Negotiable Cross-Track Rules .....                             | 7  |
| 5. VS Code Setup — Mandatory Extensions .....                          | 8  |
| 5.1 Universal Extensions (Install for BOTH Tracks) .....               | 8  |
| 5.2 Java Track — Additional Extensions .....                           | 8  |
| 5.3 Python Track — Additional Extensions .....                         | 9  |
| 5.4 Frontend (React) — Additional Extensions .....                     | 9  |
| 6. GitHub Repository & Version Control Standards .....                 | 9  |
| 6.1 Repository Setup .....                                             | 9  |
| 6.2 Branch Strategy (Simplified GitHub Flow).....                      | 9  |
| 6.3 Commit Message Convention — Conventional Commits (Mandatory) ..... | 10 |
| 6.4 Commit Cadence Policy — The Non-Negotiable .....                   | 10 |
| 6.5 Standard Folder / Module Structure .....                           | 10 |
| 6.6 Mandatory Root Files.....                                          | 11 |
| 6.7 Pull Request Discipline .....                                      | 11 |
| 7. Architecture & Design Deliverables.....                             | 12 |
| 7.1 System Architecture Diagram .....                                  | 12 |
| 7.2 Entity-Relationship (ER) Diagram .....                             | 12 |
| 7.3 Class / Module Diagram .....                                       | 12 |
| 7.4 API Contract (Swagger / OpenAPI) .....                             | 12 |
| 8. Day 11 — Review-I: MVP Checklist & Rubric .....                     | 13 |
| 8.1 Pre-Review Checklist.....                                          | 13 |
| 8.2 Review-I Acceptance Criteria .....                                 | 13 |

|                                                                                               |           |
|-----------------------------------------------------------------------------------------------|-----------|
| <b>9. Day 41 — Review-II: Full Product, Live .....</b>                                        | <b>13</b> |
| <b>9.1 Functionality .....</b>                                                                | <b>13</b> |
| <b>9.2 Testing (Basic — Realistic Target) .....</b>                                           | <b>14</b> |
| <b>9.3 CI/CD via GitHub Actions .....</b>                                                     | <b>14</b> |
| <b>9.4 Cloud Deployment .....</b>                                                             | <b>15</b> |
| <b>9.5 Security Basics .....</b>                                                              | <b>15</b> |
| <b>9.6 Basic Monitoring .....</b>                                                             | <b>16</b> |
| <b>9.7 Documentation .....</b>                                                                | <b>16</b> |
| <b>9.8 Commit Cadence .....</b>                                                               | <b>16</b> |
| <b>9.9 Review-II .....</b>                                                                    | <b>16</b> |
| <b>10. Day 60 — Review-III: Enhancement Feature .....</b>                                     | <b>16</b> |
| <b>10.1 Enhancement Menu (choose ONE, or propose your own idea by Day 43) .....</b>           | <b>16</b> |
| <b>10.2 Process &amp; Checklist .....</b>                                                     | <b>16</b> |
| <b>10.3 Commit Cadence .....</b>                                                              | <b>17</b> |
| <b>10.4 Review-III .....</b>                                                                  | <b>17</b> |
| <b>11. Definition of Done — Quality Checklist for Every PR .....</b>                          | <b>17</b> |
| <b>12. Documentation Standards .....</b>                                                      | <b>18</b> |
| <b>12.1 README.md — Exact Required Sections, In Order .....</b>                               | <b>18</b> |
| <b>12.2 Code Comment Standards .....</b>                                                      | <b>18</b> |
| <b>12.3 This Document's Own Formatting Rule (apply the same to your project report) .....</b> | <b>18</b> |
| <b>13. Overall Grading Rubric .....</b>                                                       | <b>18</b> |
| <b>13.1 Grade Bands .....</b>                                                                 | <b>18</b> |
| <b>14. Appendices .....</b>                                                                   | <b>19</b> |
| <b>Appendix A — Sample Problem Statement Bank .....</b>                                       | <b>19</b> |
| <b>Appendix B — Conventional Commit Cheat Sheet .....</b>                                     | <b>19</b> |
| <b>Appendix C — Weekly Commit Tracker Template .....</b>                                      | <b>19</b> |
| <b>Appendix D — Glossary .....</b>                                                            | <b>20</b> |
| <b>Appendix E — VS Code Extensions Quick-Install List .....</b>                               | <b>20</b> |

# 1. About This Capstone Project

---

This document is your single source of truth for the entire 60-day Capstone Project. It is written at nano-level, meaning every requirement, every deliverable, and every deadline is spelled out clearly — kindly read it fully before starting Day 1 work.

## 1.1 Purpose

To take you, in a realistic and achievable manner, through the complete Software Development Life Cycle (SDLC) — problem selection, design, development, testing, deployment, and enhancement — exactly the way it is practiced in the IT industry, adjusted sensibly to what a Semester-5 student can genuinely learn and execute in 60 days.

## 1.2 The Four Non-Negotiables

- Commit Discipline: Minimum 3 meaningful GitHub commits every week, for all 9 weeks — no exceptions (see Section 5.4).
- Three Reviews: Day 11 (Review-I — MVP), Day 41 (Review-II — Full Product Live), Day 60 (Review-III — Enhancement). Missing a review date without prior mentor approval is treated as a program failure for that phase.
- Design Documents: Architecture Diagram, ER Diagram, and Class/Module Diagram are mandatory — stored inside /docs folder of your repository.
- Live Product: By Day 41, your project must be running on a public URL that anyone (including your external examiner) can open in a browser — not just “works on my laptop.”

## 1.4 Roles

| Role                               | Responsibility                                                                                                                                   |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Student (You)                      | Sole developer of the project: coding, testing, documentation, deployment, and explaining your own design decisions confidently at every review. |
| Guide / Mentor                     | Reviews your GitHub commits weekly, evaluates each review document, and gives you clear, written feedback.                                       |
| Review Panel / HOD (if applicable) | Faculty members who may sit in on Day 11 / 41 / 60 demonstrations and score.                                                                     |

## 2. Master Timeline — Day 1 to Day 60

The 60 days are divided into 3 phases, anchored by 3 review dates. Section 2.2 further breaks this into 9 weekly sprints so that tracking your weekly commit target becomes very simple.

### 2.1 Phase Map

| Phase      | Days        | Focus                          | Key Deliverable                                                        |
|------------|-------------|--------------------------------|------------------------------------------------------------------------|
| Phase 0    | Day 1       | Problem Statement Finalization | Problem_Statement.md finalized & committed to GitHub                   |
| Phase 1    | Day 2 – 10  | Design & Planning              | Architecture, ER, Class diagrams; repo setup; DB schema; VS Code setup |
| REVIEW-I   | Day 11      | MVP Demonstration              | Working MVP demo + $\geq 6$ commits across 2 weeks                     |
| Phase 2    | Day 12 – 40 | Full Development               | All modules complete, basic tests, CI/CD via GitHub Actions            |
| REVIEW-II  | Day 41      | Full Product — Live            | Cloud-hosted product, working CI/CD pipeline, basic security           |
| Phase 3    | Day 42 – 59 | Enhancement Feature            | One new feature (AI or otherwise) designed, built, and deployed        |
| REVIEW-III | Day 60      | Final Review                   | Enhancement live on production + final viva/demo + grading             |

### 2.2 Weekly Sprint Breakdown (9 Sprints)

| Sprint | Days      | Focus                                                                                          | Min. Commits |
|--------|-----------|------------------------------------------------------------------------------------------------|--------------|
| Week 1 | Day 1–7   | Problem statement, domain study, stack decision, GitHub repo + VS Code setup                   | $\geq 3$     |
| Week 2 | Day 8–14  | Architecture/ER/Class diagrams, DB schema, project scaffolding, MVP build. REVIEW-I on Day 11. | $\geq 3$     |
| Week 3 | Day 15–21 | Core backend APIs + frontend integration for main modules                                      | $\geq 3$     |
| Week 4 | Day 22–28 | Login/signup (auth), remaining CRUD modules, start unit tests                                  | $\geq 3$     |
| Week 5 | Day 29–35 | Finish unit tests, set up GitHub Actions CI pipeline                                           | $\geq 3$     |
| Week 6 | Day 36–42 | Cloud deployment via GitHub Actions, basic security check. REVIEW-II on Day 41.                | $\geq 3$     |
| Week 7 | Day 43–49 | Enhancement idea research + proposal + small proof-of-concept                                  | $\geq 3$     |
| Week 8 | Day 50–56 | Enhancement full build + tests + integrate into live product                                   | $\geq 3$     |

| Sprint | Days      | Focus                                                                 | Min. Commits   |
|--------|-----------|-----------------------------------------------------------------------|----------------|
| Week 9 | Day 57–60 | Final polish, demo video, documentation freeze. REVIEW-III on Day 60. | ≥ 2 (prorated) |

**Program-wide minimum commit floor: 26 commits in 60 days. This is a MINIMUM, kindly aim to do more — real developers commit small, working changes very frequently.**

### 3. Day 1 — Problem Statement Finalization

Day 1 has exactly ONE deliverable: a finalized, committed, Problem\_Statement.md file.

#### 3.1 Problem Statement Selection

- Please refer **R2021\_Sem5\_Computing\_Project\_Students.xlsx**
- Select 1 project out of 128 projects listed in the excel file.

#### 3.2 Problem Statement Template (copy into Problem\_Statement.md)

```
# Problem Statement

## 1. Title
## 2. Domain
## 3. Who is the user? (2-3 user types, with roles)
## 4. What problem are we solving? (3-5 sentences, real-life example)
## 5. Proposed Solution (what the application will do, feature-wise)
## 6. Core Entities / Database Tables (list all, minimum 5)
## 7. User Roles & Permissions (minimum 2 distinct roles, e.g. Admin & User)
## 8. Success Criteria (e.g. 'a user should be able to book an appointment in under 1 minute')
## 9. Out of Scope (clearly list what you will NOT build, to avoid over-commitment)
## 10. Chosen Track: Java (Spring Boot) / Python (Django or FastAPI)
```

#### 3.3 Acceptance Criteria

| Criterion                                                                   |
|-----------------------------------------------------------------------------|
| Has real business logic (not just Add/View/Edit/Delete)                     |
| ≥ 5 database tables with real relationships between them                    |
| Requires ≥ 2 different user roles (e.g. Admin & Student)                    |
| Has scope for one 3rd-party integration (email, SMS, maps, payment-sandbox) |
| Has natural scope for adding an AI feature later (Day 41–60)                |
| Original enough — not a copy-paste YouTube tutorial clone                   |

### 3.4 Straight Rejection List

- Single-table to-do / notes apps with no login system.
- Direct unmodified clones (“build Instagram” / “build Swiggy”) with zero originality.
- Projects with fewer than 5 real database tables or a single user role.
- Pure static/informational websites with no real backend logic.

## 4. Tech Stack

Kindly choose ONE track for the full 60 days as per the placement training undergone.

| Layer                     | Java Track                                                   | Python Track                                                 |
|---------------------------|--------------------------------------------------------------|--------------------------------------------------------------|
| Frontend                  | React.js (JavaScript) + Bootstrap / Tailwind CSS + Axios     | React.js (JavaScript) + Bootstrap / Tailwind CSS + Axios     |
| Backend Framework         | Spring Boot 3.x (Java 17)                                    | Django REST Framework OR FastAPI (Python 3.10+)              |
| Auth                      | Spring Security + JWT (jjwt library)                         | Django Auth / SimpleJWT (DRF) or FastAPI-Users / python-jose |
| ORM / Data Layer          | Spring Data JPA + Hibernate                                  | Django ORM OR SQLAlchemy                                     |
| Database                  | MySQL 8 OR PostgreSQL 15                                     | MySQL 8 OR PostgreSQL 15                                     |
| Build Tool                | Maven                                                        | pip + requirements.txt (Poetry optional)                     |
| Testing (basic)           | JUnit 5 (unit tests only, mandatory)                         | Pytest (unit tests only, mandatory)                          |
| API Docs                  | springdoc-openapi (Swagger UI) — auto-generated, easy to add | drf-yasg (DRF) or built-in FastAPI Swagger — auto-generated  |
| CI/CD                     | GitHub Actions (mandatory — see Section 8.3)                 | GitHub Actions (mandatory — see Section 8.3)                 |
| Containerization (Docker) | OPTIONAL BONUS — not compulsory for grading                  | OPTIONAL BONUS — not compulsory for grading                  |
| Backend Hosting           | Render or Railway (free tier)                                | Render or Railway (free tier)                                |
| Frontend Hosting          | Vercel or Netlify (free tier)                                | Vercel or Netlify (free tier)                                |
| Managed DB Hosting        | Railway MySQL/Postgres, Clever Cloud, or Aiven (free tier)   | Railway MySQL/Postgres, Clever Cloud, or Aiven (free tier)   |

### 4.1 Non-Negotiable Cross-Track Rules

- REST API only, organized under /api/ (versioning like /api/v1/ is a good practice but optional here).
- Every API response should follow one consistent JSON structure across your whole project, e.g. { success, data, message }.

- Use correct HTTP status codes: 200/201 for success, 400/401/404 for client errors, 500 for server errors.
- Passwords must always be hashed (bcrypt) — never stored or printed in plain text.
- All secrets/passwords/API keys go into a .env file — never hardcoded, never committed to GitHub.

## 5. VS Code Setup — Mandatory Extensions

Visual Studio Code (VS Code) is the required IDE for this capstone project. Kindly install VS Code from [code.visualstudio.com](https://code.visualstudio.com) and set up the following extensions BEFORE Day 2. This ensures your codebase is built with proper linting, formatting, and Git integration from day one — exactly like a real development team's setup.

### 5.1 Universal Extensions (Install for BOTH Tracks)

| Extension                                       | Extension ID                      | Why You Need It                                                                                |
|-------------------------------------------------|-----------------------------------|------------------------------------------------------------------------------------------------|
| GitLens — Git Supercharged                      | eamodio.gitlens                   | See commit history, blame, and authorship inline — helps you track your own commit discipline. |
| Prettier — Code Formatter                       | esbenp.prettier-vscode            | Auto-formats JS/React/JSON/CSS files on save.                                                  |
| ESLint                                          | dbaeumer.vscode-eslint            | Flags JavaScript/React code quality issues live.                                               |
| Thunder Client                                  | rangav.vscode-thunder-client      | Test your REST APIs directly inside VS Code — lightweight Postman alternative.                 |
| GitHub Actions                                  | github.vscode-github-actions      | View and edit your CI/CD workflow files with syntax highlighting.                              |
| GitHub Pull Requests and Issues                 | github.vscode-pull-request-github | Create and review PRs without leaving VS Code.                                                 |
| EditorConfig for VS Code                        | editorconfig.editorconfig         | Keeps formatting consistent across your whole team/repo.                                       |
| Error Lens                                      | usernamehw.errorlens              | Shows errors/warnings inline instead of only in the Problems tab.                              |
| Todo Tree                                       | gruntfuggly.todo-tree             | Tracks all your // TODO comments in one place.                                                 |
| Docker (optional, only if attempting the bonus) | ms-azuretools.vscode-docker       | Needed only if you choose the optional Docker bonus (Section 4).                               |

### 5.2 Java Track — Additional Extensions

| Extension               | Extension ID             | Why You Need It                                                                       |
|-------------------------|--------------------------|---------------------------------------------------------------------------------------|
| Extension Pack for Java | vscjava.vscode-java-pack | Bundles Language Support for Java, Debugger, Test Runner, Maven, and Project Manager. |

| Extension                  | Extension ID                | Why You Need It                                                          |
|----------------------------|-----------------------------|--------------------------------------------------------------------------|
| Spring Boot Extension Pack | vmware.vscode-boot-dev-pack | Spring Boot Dashboard, Initializr, and live application info.            |
| Lombok Annotations Support | gabrielbb.vscode-lombok     | Required if you use Lombok (@Getter/@Setter) to reduce boilerplate code. |

### 5.3 Python Track — Additional Extensions

| Extension          | Extension ID              | Why You Need It                                                         |
|--------------------|---------------------------|-------------------------------------------------------------------------|
| Python (Microsoft) | ms-python.python          | Core Python language support, debugging, virtual environment detection. |
| Pylance            | ms-python.vscode-pylance  | Fast, accurate IntelliSense and type checking for Python.               |
| Black Formatter    | ms-python.black-formatter | Auto-formats Python code on save to PEP-8 standard.                     |
| autoDocstring      | njpwerner.autodocstring   | Auto-generates PEP-257 docstring templates for functions/classes.       |

### 5.4 Frontend (React) — Additional Extensions

| Extension                              | Extension ID                    | Why You Need It                                                     |
|----------------------------------------|---------------------------------|---------------------------------------------------------------------|
| ES7+ React/Redux/React-Native snippets | dsznajder.es7-react-js-snippets | Fast component/hook boilerplate with short prefixes (e.g. rafce).   |
| Tailwind CSS IntelliSense              | bradlc.vscode-tailwindcss       | Autocomplete for Tailwind utility classes (only if using Tailwind). |

*How to install: open VS Code → press Ctrl+Shift+X (Extensions panel) → search the exact name or paste the Extension ID in the search box → click Install. A ready-to-use list is also provided in Appendix E for quick copy-paste.*

## 6. GitHub Repository & Version Control Standards

### 6.1 Repository Setup

- Repo name: kebab-case, descriptive (e.g. campus-leave-management-system), not “capstone-project” or “test-repo”.
- Visibility: Public, so it becomes a genuine portfolio piece for placements/internships.
- Turn ON branch protection for main from Day 2: require Pull Request + passing CI check before merge.

### 6.2 Branch Strategy (Simplified GitHub Flow)

| Branch               | Purpose                                                                |
|----------------------|------------------------------------------------------------------------|
| main                 | Always working/deployable. Protected. No direct pushes after Day 2.    |
| feature/<short-desc> | One branch per feature, e.g. feature/user-login, feature/order-module. |
| fix/<short-desc>     | For bug fixes found after a feature is merged.                         |

Note: A separate “develop” branch is OPTIONAL in this realistic edition — feature branches merging straight into main (after CI passes) is perfectly acceptable for a solo capstone project.

### 6.3 Commit Message Convention — Conventional Commits (Mandatory)

Format: <type><scope>: <short description> — all lowercase, present tense, no full stop at the end.

| Type     | Meaning                              | Example                                      |
|----------|--------------------------------------|----------------------------------------------|
| feat     | New feature                          | feat(auth): add login endpoint               |
| fix      | Bug fix                              | fix(orders): correct total price calculation |
| docs     | Documentation only                   | docs(readme): add setup instructions         |
| test     | Adding/fixing tests                  | test(orders): add unit test for checkout     |
| refactor | Code restructure, no behavior change | refactor(service): extract pricing logic     |
| chore    | Tooling, dependency, config change   | chore(deps): update spring-boot version      |
| ci       | CI/CD workflow changes               | ci(actions): add test step to workflow       |

**BANNED commit messages (auto-loss of Git hygiene marks): “fix”, “update”, “changes”, “wip”, “final”, “final final”, “asdf”, or empty messages.**

### 6.4 Commit Cadence Policy — The Non-Negotiable

- Minimum 3 commits every calendar week (Monday to Sunday), for all 9 sprint weeks.
- Anti-gaming rule: all 3 commits must NOT be made within a single 24-hour window. Kindly spread them across at least 2 different days each week.
- Each commit should be one logical, working change — not one giant commit dumping 10 files at the deadline.
- Guide/Mentor checks the commit graph every Sunday against the tracker in Appendix C.
- 2 consecutive weeks below the floor will result in a mandatory catch-up discussion with your guide.

### 6.5 Standard Folder / Module Structure

Both tracks must be modular — clear separation between API/controller layer, business logic (service), data access (repository), and models. Kindly do not write all logic inside one giant file.

**Java (Spring Boot) — example folder tree**

```

src/main/java/com/college/appname/
├── config/           (SecurityConfig, SwaggerConfig, CorsConfig)
├── controller/       (REST endpoints – thin, no business logic here)
├── service/          (business logic)
├── repository/       (Spring Data JPA interfaces)
├── model/entity/     (JPA entities)
├── dto/              (request/response objects)
├── exception/        (custom exceptions + error handler)
src/test/java/...    (unit tests, mirrors main structure)
docs/                (diagrams, API contract)
.env.example, README.md, pom.xml

```

### Python (FastAPI / Django) — example folder tree

```

app/
├── api/              (routers/views – thin, no business logic here)
├── core/              (settings, security helpers)
├── models/            (ORM models)
├── schemas/           (Pydantic schemas / DRF serializers)
├── services/          (business logic)
tests/                (unit tests, mirrors app structure)
docs/                 (diagrams, API contract)
.env.example, README.md, requirements.txt

```

## 6.6 Mandatory Root Files

- README.md (see Section 12.1 for the exact required sections)
- .gitignore (never commit .env, /node\_modules, /target, /\_\_pycache\_\_, /venv)
- LICENSE (MIT recommended for a portfolio-friendly project)
- .env.example (list every required variable name, no real secret values)
- CHANGELOG.md (simple log of what changed at each review, updated at Day 11 / 41 / 60)

## 6.7 Pull Request Discipline

- Every feature is merged via a Pull Request, even though you are working solo — no direct pushes to main after Day 2.
- PR description should mention: what changed, why, and how you tested it.
- Use GitHub Issues + a simple Project board (To Do → In Progress → Done) to track your own tasks — looks great to reviewers and recruiters both.

## 7. Architecture & Design Deliverables

---

These 4 documents are due before Day 11 (version 1) and must be updated at Day 41 (as-built) and Day 60 (with your enhancement added). All are stored inside /docs/diagrams/ in your repository and linked from your README. This is what separates a real engineering project from “just some code that runs.”

### 7.1 System Architecture Diagram

Tool suggestion: draw.io (free, works in browser) or Excalidraw. Export as PNG and keep the editable source file too.

Must show, at minimum:

- Client layer (your React frontend, running in the browser)
- Backend/API layer (your Spring Boot or Django/FastAPI application)
- Database layer (MySQL/PostgreSQL)
- Any external service used (email service, payment sandbox, maps, AI API, etc.)
- Hosting boundary (which platform hosts what — e.g. Vercel for frontend, Render for backend)

### 7.2 Entity-Relationship (ER) Diagram

- Minimum 5 entities/tables (matches your Problem Statement, Section 6).
- Show Primary Key (PK) and Foreign Key (FK) clearly on every table.
- Show correct relationship type between tables: One-to-Many, Many-to-Many (with a junction table shown), or One-to-One.
- Tool suggestion: dbdiagram.io (free, very beginner-friendly) — export PNG + the .dbml source file.

### 7.3 Class / Module Diagram

- Java track: simple UML class diagram of your core entity classes, service classes, and how they relate (which service uses which repository, etc.).
- Python track: a module diagram showing your core models and service functions/classes and how they depend on each other.
- This must match your ACTUAL code — your guide may open your repo and check.

### 7.4 API Contract (Swagger / OpenAPI)

- Use the auto-generated Swagger docs from springdoc-openapi (Java) or drf-yasg/FastAPI (Python) — this needs very little extra effort since it reads your code directly.
- Hosted live at /swagger-ui.html or /docs on your deployed backend by Day 41.
- Every endpoint should clearly show: method, path, whether login is required, request format, and response format.

## 8. Day 11 — Review-I: MVP Checklist & Rubric

---

Goal: prove that your basic skeleton works end-to-end for at least 2 core flows — the focus here is “does it actually work,” not “does it look perfect.”

### 8.1 Pre-Review Checklist

- ☐ Problem\_Statement.md finalized, committed, and guide-approved
- ☐ Architecture Diagram v1 committed to /docs/diagrams/
- ☐ ER Diagram v1 committed to /docs/diagrams/
- ☐ Class/Module Diagram v1 committed to /docs/diagrams/
- ☐ Repository follows the standard folder structure (Section 6.5)
- ☐ README v1 present: project overview, tech stack, how to run it locally
- ☐ ≥ 6 commits logged across Weeks 1–2 (≥ 3/week), Conventional Commits format followed
- ☐ Login/Signup module working, issuing a real JWT (or session) on successful login
- ☐ ≥ 2 core modules functional end-to-end (frontend → backend → database → back to frontend)
- ☐ App runs locally by following ONLY the README instructions (your guide will actually test this)

### 8.2 Review-I Acceptance Criteria

| Criterion                                   |
|---------------------------------------------|
| Problem clarity & scope discipline          |
| Architecture & diagram quality/accuracy     |
| Code quality & modularity                   |
| Working functionality (2+ end-to-end flows) |
| Git hygiene (commits, messages, branches)   |

## 9. Day 41 — Review-II: Full Product, Live

---

Goal: your product is feature-complete as per Problem\_Statement.md, hosted on a public URL, tested with basic automated tests, and secured to a sensible baseline — all achievable in 60 days.

### 9.1 Functionality

- ☐ 100% of the features listed in Problem\_Statement.md are implemented
- ☐ Login system supports at least 2 different user roles (e.g. Admin & User), with role-based screens/permissions

- ☐ All planned CRUD + business-logic modules complete and match your ER diagram

## 9.2 Testing (Basic — Realistic Target)

- ☐ Unit tests written for the business-logic (service) layer — target  $\geq 40\%$  of service methods covered
- ☐ At least one test class per major module (e.g. UserServiceTest, OrderServiceTest)
- ☐ All written tests pass — zero failing tests allowed in the submitted code
- ☐ OPTIONAL BONUS: integration tests using an in-memory/test database, or a basic Selenium/Cypress test for one user flow

## 9.3 CI/CD via GitHub Actions

### 9.3.1 What Your Pipeline Must Do

- ☐ Trigger automatically on every push / pull request to main
- ☐ Step 1 — Checkout code
- ☐ Step 2 — Install dependencies (Maven for Java / pip for Python, npm for React)
- ☐ Step 3 — Run linter (basic style check)
- ☐ Step 4 — Run all unit tests — pipeline must FAIL (turn red) if any test fails
- ☐ Step 5 (only on push to main, after tests pass) — trigger deployment to your hosting platform

### 9.3.2 Sample Workflow — Backend (Java, Spring Boot) — .github/workflows/backend.yml

```
name: Backend CI/CD
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin' }
      - name: Build and Run Tests
        run: mvn -B clean verify
  deploy:
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - name: Trigger Render Deploy Hook
        run: curl -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}"
```

### 9.3.3 Sample Workflow — Backend (Python, FastAPI/Django) — .github/workflows/backend.yml

```
name: Backend CI/CD
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```

```

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run Tests
        run: pytest
  deploy:
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - name: Trigger Render Deploy Hook
        run: curl -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}"

```

### 9.3.4 Sample Workflow — Frontend (React) — .github/workflows/frontend.yml

```

name: Frontend CI/CD
on:
  push:
    branches: [main]
jobs:
  build-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm install
      - run: npm run build
      - name: Deploy to Vercel
        run: npx vercel --prod --token=${{ secrets.VERCEL_TOKEN }} --yes

```

**Note:** *Render/Railway/Vercel deploy hook URLs and tokens must be stored as GitHub Secrets (Repo → Settings → Secrets and variables → Actions) — never pasted directly into the YAML file.*

## 9.4 Cloud Deployment

- ☐ Backend hosted on a public URL (Render or Railway free tier)
- ☐ Frontend hosted on a public URL (Vercel or Netlify)
- ☐ Database is a cloud-hosted instance (Railway/Clever Cloud/Aiven free tier) — not a local-only database
- ☐ All secrets/config values set as environment variables on the hosting platform — zero secrets committed to GitHub (double-check .env is in .gitignore)

## 9.5 Security Basics

- ☐ Passwords hashed with bcrypt — never plain text, never logged to console
- ☐ All database queries go through your ORM (JPA/Hibernate or Django ORM/SQLAlchemy) — no raw string-concatenated SQL, which prevents SQL injection automatically
- ☐ Basic server-side input validation on all forms (e.g. @Valid in Spring, serializers in DRF, Pydantic in FastAPI)

☐ CORS configured properly — not left wide open with \* in a way that ignores security entirely

## 9.6 Basic Monitoring

- ☐ A simple /health or /api/health endpoint that returns “OK” / service status
- ☐ Basic console/file logging of key events (user signup, login, errors) — structured logging tools are OPTIONAL BONUS only

## 9.7 Documentation

- ☐ README v2: adds live demo link, architecture diagram, API docs link, and deployment steps
- ☐ All 4 design artifacts (Section 7) updated to reflect the as-built system
- ☐ CHANGELOG.md updated for this release

## 9.8 Commit Cadence

- ☐ ≥ 3 commits/week maintained for Weeks 3–6 (≥ 12 commits this phase, ≥ 18 cumulative)

## 9.9 Review-II

| Criterion                                          |
|----------------------------------------------------|
| Functionality & completeness vs. Problem Statement |
| Code quality & modularity                          |
| Testing (basic unit tests) + working CI pipeline   |
| Deployment: GitHub Actions CD + live hosting       |
| Security basics & documentation                    |

# 10. Day 60 — Review-III: Enhancement Feature

---

Goal: prove you can safely add a new feature to an already-running product — exactly what a junior developer does in their first project at a company.

## 10.1 Enhancement Menu (choose ONE, or propose your own idea by Day 43)

- Please refer **R2021\_Sem5\_Computing\_Project\_Students.xlsx** (Column D, E, F as per department. ECE/EEE should follow CSE department specialization)
- The chosen project specialization only should be implemented

## 10.2 Process & Checklist

- ☐ Enhancement\_Proposal.md (problem, solution approach, tech choice) committed by Day 45
- ☐ Feature built on its own feature branch, merged via Pull Request after testing
- ☐ At least 1-2 new unit tests written specifically for the enhancement
- ☐ Enhancement deployed to the SAME live product (not a separate demo-only app)
- ☐ Architecture diagram updated to show the new component/integration
- ☐ README v3 (final): polished and complete — this becomes your portfolio/resume project now
- ☐ 2–4 minute demo video recorded (phone screen-recorder or free tool like Loom) and linked in README
- ☐ CHANGELOG.md final entry added

### 10.3 Commit Cadence

- ☐  $\geq 3$  commits/week for Weeks 7–8,  $\geq 2$  for the partial Week 9 — cumulative program total  $\geq 26$  commits

### 10.4 Review-III

| Criterion                                             |
|-------------------------------------------------------|
| Relevance & usefulness of the enhancement             |
| Technical implementation quality                      |
| Clean integration into the existing live system       |
| Testing & documentation of the enhancement            |
| Demo quality & your ability to explain it confidently |

## 11. Definition of Done — Quality Checklist for Every PR

Apply this checklist to EVERY pull request, at every phase, not just at review dates. If any box is unticked, kindly do not merge yet.

- ☐ Code self-reviewed against this checklist before merging
- ☐ No hardcoded secrets, passwords, or API keys anywhere in the code
- ☐ No leftover debug print statements (System.out.println / print / console.log) in submitted code
- ☐ All existing and new tests pass — no red/failing build merged into main
- ☐ No errors shown in browser console; no unhandled crashes during manual testing
- ☐ UI checked at least on both mobile and desktop screen widths
- ☐ Commit messages follow Conventional Commits (Section 6.3)
- ☐ Merged only via Pull Request — never a direct push to main
- ☐ README updated if the change affects setup, features, or usage

## 12. Documentation Standards

---

### 12.1 README.md — Exact Required Sections, In Order

1. Project Title + one-line tagline
2. Live Demo link + Video Demo link (from Day 41 onward)
3. Overview (2-4 sentences on what problem it solves)
4. Architecture Diagram (embedded image)
5. Tech Stack (table, matches Section 4 of this document)
6. Features (bulleted list, grouped module-wise)
7. Screenshots of key screens
8. Getting Started: prerequisites, clone, install, env vars, run locally
9. Environment Variables table (name, description, required Y/N)
10. API Documentation link (hosted Swagger)
11. Running Tests (exact command to run)
12. Deployment (how it is deployed, which platform, CI/CD summary)
13. Folder Structure (short tree)
14. Future Enhancements
15. License
16. Author / Contact

### 12.2 Code Comment Standards

- Java: JavaDoc comment on every public class and public method — purpose, @param, @return.
- Python: docstring (PEP-257 style) on every module, class, and public function — purpose, Args, Returns.
- Comments should explain WHY a decision was made, not narrate WHAT the code obviously already shows.

### 12.3 This Document's Own Formatting Rule (apply the same to your project report)

- Header: kept blank/nil — avoids a cluttered repeated title on every page.
- Footer: page number only, centered.
- Headings numbered and consistent, so a Table of Contents is meaningful.
- Body text 11–12pt, comfortable line spacing, one consistent font throughout.

## 13. Overall Grading Rubric

---

| Component                                                         | Weight |
|-------------------------------------------------------------------|--------|
| Review-I — Day 11 (MVP)                                           | 20%    |
| Review-II — Day 41 (Full Product, Live)                           | 45%    |
| Review-III — Day 60 (Enhancement)                                 | 25%    |
| Git Hygiene & Process (commit cadence, PR discipline, throughout) | 10%    |

### 13.1 Grade Bands

| Score      | Meaning                                                          |
|------------|------------------------------------------------------------------|
| 8.5 – 10.0 | Gold Standard — placement/internship-ready portfolio project     |
| 7.0 – 8.4  | Strong — minor gaps only, easily fixable                         |
| 5.0 – 6.9  | Needs Improvement — clear gaps in testing/deployment/design docs |
| Below 5.0  | Not Ready — catch-up sprint required with guide's help           |

## 14. Appendices

### Appendix A — Sample Problem Statement Bank

| Domain     | One-Liner                                   | Sample Tables                                                    |
|------------|---------------------------------------------|------------------------------------------------------------------|
| EdTech     | Online quiz & exam system with auto-grading | 6 tables: Student, Teacher, Quiz, Question, Attempt, Result      |
| HealthTech | Clinic appointment booking with reminders   | 5 tables: Patient, Doctor, Appointment, Slot, Prescription       |
| HRTech     | Campus placement drive tracker              | 6 tables: Student, Company, Drive, Application, Interview, Offer |
| Logistics  | Canteen food pre-order system               | 5 tables: Student, MenuItem, Order, OrderItem, Payment           |
| Community  | Hostel complaint & maintenance tracker      | 5 tables: Student, Warden, Complaint, Category, Status           |

### Appendix B — Conventional Commit Cheat Sheet

```

feat(scope): add new feature
fix(scope): patch a bug
docs(scope): documentation change only
test(scope): add or fix a test
refactor(scope): restructure code, no behavior change
chore(scope): tooling/dependency/config change
ci(scope): CI/CD workflow change

Example real history:
feat(auth): implement JWT login endpoint
test(auth): add unit test for login service
fix(auth): handle expired token edge case
docs(readme): document environment variables

```

### Appendix C — Weekly Commit Tracker Template

| Wk | Date Range | Target | Actual | PR Links / Notes | Sign-off |
|----|------------|--------|--------|------------------|----------|
| 1  |            | ≥ 3    |        |                  |          |
| 2  |            | ≥ 3    |        |                  |          |
| 3  |            | ≥ 3    |        |                  |          |
| 4  |            | ≥ 3    |        |                  |          |
| 5  |            | ≥ 3    |        |                  |          |
| 6  |            | ≥ 3    |        |                  |          |
| 7  |            | ≥ 3    |        |                  |          |
| 8  |            | ≥ 3    |        |                  |          |
| 9  |            | ≥ 2    |        |                  |          |

## Appendix D — Glossary

| Term              | Definition                                                                                              |
|-------------------|---------------------------------------------------------------------------------------------------------|
| MVP               | Minimum Viable Product — the smallest version that proves the core idea works end-to-end                |
| CI/CD             | Continuous Integration / Continuous Deployment — automated build, test, and release pipeline            |
| ORM               | Object-Relational Mapper — lets you work with database rows as code objects (JPA/Hibernate, Django ORM) |
| JWT               | JSON Web Token — a signed token used to identify a logged-in user on each API request                   |
| Swagger / OpenAPI | A standard, auto-generated, browsable documentation page for your REST APIs                             |
| Deploy Hook       | A special URL that, when called, tells your hosting platform to pull the latest code and redeploy       |
| ER Diagram        | Entity-Relationship Diagram — a visual map of your database tables and how they connect                 |

## Appendix E — VS Code Extensions Quick-Install List

Kindly copy the relevant Extension IDs below and paste each into the VS Code Extensions search bar (Ctrl+Shift+X) to install quickly.

Universal: eamodio.gitlens, esbenp.prettier-vscode, dbaeumer.vscode-eslint, rangav.vscode-thunder-client, github.vscode-github-actions, github.vscode-pull-request-github, editorconfig.editorconfig, usernamehw.errorlens, gruntfuggly.todo-tree

Java Track: vscjava.vscode-java-pack, vmware.vscode-boot-dev-pack, gabrielbb.vscode-lombok

Python Track: ms-python.python, ms-python.vscode-pylance, ms-python.black-formatter, njpwerner.autodocstring

|           |                                                                                       |
|-----------|---------------------------------------------------------------------------------------|
| Frontend: | <code>dsznajder.es7-react-js-snippets</code> , <code>bradlc.vscode-tailwindcss</code> |
| Optional: | <code>ms-azuretools.vscode-docker</code> (only for Docker bonus)                      |